The Absolute Beginner's Guide to WebSockets

Welcome! If you've ever wondered how chat apps, live sports scores, or multiplayer games work without you refreshing the page, the answer is usually **WebSockets**.

This guide uses examples from your actual project (a "Jam Room" music sharing app) to show how it all works.

---

1. WebSocket vs HTTP (The "Why")

The HTTP Way (Restaurant Style)

Imagine you are at a restaurant. You (the **Client**) ask the waiter (the **Server**) for a burger. The waiter goes to the kitchen, gets the burger, gives it to you, and then **leaves**. If you want a napkin, you have to call the waiter back and ask again.

- **One-way:** Client asks, Server responds.
- **Heavy:** Every time you ask, you have to say "Table 5, I'm the guy in the red hat" (headers/cookies).
- **Not Real-time:** The server cannot give you a burger unless you ask for it first.

The WebSocket Way (Phone Call Style)

Imagine you are on a phone call. You and the server have an **open line**. You can talk, they can talk, or you can both talk at the same time. The line stays open until one of you hangs up.

- **Two-way (Full Duplex):** Both can send messages at any time.
- **Lightweight:** Once the "call" is connected, you don't need to repeat who you are every time.
- **Real-time:** The server can say "Hey, someone joined the room!" without you asking.

The Handshake (The "Wait, can we talk?")

A WebSocket doesn't start as a WebSocket. It starts as an **HTTP request** (the standard way web pages load). It basically says: "*Hey Server, I'd like to stop being a visitor and start a permanent conversation.*"

1. **Client:** "Hi Server, I'm at `http://localhost:8080/ws`. Can we upgrade this to a WebSocket?"
2. **Server:** "I check my rules... Yes! I see you are allowed. Let's switch."
3. **Connection:** The HTTP request ends, and a persistent **TCP tunnel** (think of it as a private, direct pipe between you and the server) is created.

---

3. The 6 Pillars of WebSocket (The Toolbox)

In plain JavaScript, you use a `WebSocket` object. Imagine these are the 'buttons' and 'lights' on your phone:

1. `onopen` (The "Green Light")

This light turns on as soon as the connection is successful. It's the moment you know you can start talking.

```
socket.onopen = () => console.log("The connection is live!");
```

2. `onmessage` (The "Incoming Mail")

This is the most important part. It triggers every time the server sends data to you. You don't have to ask; it just arrives.

```
socket.onmessage = (event) => {
    console.log("Server sent some data: " + event.data);
};
```

3. `send()` (The "Outbox")

Use this to push data to the server. You can send text, numbers, or even files.

```
socket.send("Hey Server, play the next song!");
```

4. `onerror` (The "Warning Bell")

Triggered if something goes wrong, like a bad internet connection or a server crash.

```
socket.onerror = (error) => console.log("Something went wrong: ", error);
```

5. `onclose` (The "Disconnected" status)

Triggered when the line is cut (e.g., you closed the tab or the server shut down).

```
socket.onclose = () => console.log("The pipe is closed.");
```

6. `close()` (The "Hang up")

You use this when you want to manually end the connection and stop using resources.

```
socket.close();
```

---

4. Real Project Example: Talking to the Backend

In your "Jam Room" project, we use a library called **STOMP**. It's like "WebSocket with a structure."

The Backend (Spring Boot)

The server listens for specific "topics." In `RoomController.java`:

```
@MessageMapping("/room/{roomId}/join") // Listen for this address
@SendTo("/topic/room/{roomId}")        // Shout the answer here
public Set<String> joinRoom(...) {
    return jamSessionService.addUserToRoom(...);
}
```

The Frontend (React + STOMP)

Your `App.jsx` handles the connection. Here is a simplified version:

```
// 1. Setup the connection
const client = new Client({
```

```
  brokerURL: 'ws://localhost:8080/ws',
  onConnect: () => {
    console.log('Connected!');

    // 2. Subscribe (onmessage equivalent)
    // We listen to the "room" topic to see who is joining
    client.subscribe('/topic/room/123', (message) => {
      const users = JSON.parse(message.body);
      console.log("Current users: ", users);
    });
  }
});

// 3. Send data (send equivalent)
const joinRoom = (username) => {
  client.publish({
    destination: '/app/room/123/join',
    body: JSON.stringify({ username: username })
  });
};

client.activate(); // Start the connection
```

---

5. Summary Checklist

- [ ] **HTTP** is for requests; **WebSocket** is for conversations.
- [ ] **Handshake** is the upgrade from HTTP to WebSocket.
- [ ] **Frontend** uses `onmessage` to listen and `send()` to talk.
- [ ] **Backend** uses `@MessageMapping` to handle incoming data and `@SendTo` to broadcast to everyone.

Now you're ready to build real-time apps!